



Why Repo Spine

Repo spine is what turns a project from a loose collection of files into a real working system.

Repeatable

A project is **repeatable** when you or someone else can run it again and get the same workflow and roughly the same result.

The repo spine helps this happen because:

- Data lives in known places
- Preprocessing code is saved in src/
- Configs record settings
- Runs record metrics and parameters
- Environment files record dependencies
- Scripts like train.py define how to rerun training

Without that structure, students often forget:

- Which dataset version they used
- Which features they dropped
- Which notebook produced the result
- Which hyperparameters gave the best score

So “repeatable” means:

The work can be rerun without guessing.

Auditable

A project is **auditable** when someone can inspect it and understand:

- What was done
- Why it was done
- What data and settings were used
- What results came from which run

The repo spine makes this possible because it leaves evidence:

- **configs/** shows chosen settings
- **runs/** shows params and metrics
- **reports/** explains reasoning
- **src/** shows the real code used
- versioning links data, code, and model artefacts

So if someone asks:

- Why did this model get selected?
- Which feature set was used?
- Did this run use the same split as the last one?
- Was this feature removed because of leakage?

You can actually trace the answer.

So “auditable” means:

The project can be inspected and verified.



Scalable

A project is **scalable** when it can grow without becoming chaotic.

At the beginning, a student may have:

- One dataset
- One notebook
- One model

But later they may have:

- Multiple feature sets
- Several models
- Many runs
- Evaluation reports
- Packaged artifacts

The repo spine helps because it gives each part a place:

- Exploration stays in notebooks/
- Reusable logic moves to src/
- Settings live in configs/
- Experiment outputs go to runs/
- Reports go to reports/

So as the project expands, students do not end up with:

- Ten messy notebooks
- Random files named final_v3_latest_reallyfinal
- No idea which result is current

So “scalable” means:

The project can grow in complexity while staying organised.

Simple Example Without Repo Spine

A student says:

- “The best model is in this notebook.”
- “I think I used the cleaned CSV from last week.”
- “I’m not sure which seed I used.”
- “The chart is right, but I forgot how I made it.”

That project is not repeatable, auditable, or scalable.

With Repo Spine

A student says:

- “The model was trained with train.py using this config.”
- “The preprocessing is in `src/features/preprocess.py`.”
- “The run is logged in `runs/run_014/metrics.json`.”
- “The reasoning is summarised in `reports/model_card_v1.md`.”

That project is repeatable, auditable, and scalable.



In One Sentence

“It is what makes the project repeatable, auditable, and scalable” means:

The repo spine gives the project enough structure that it can be rerun reliably, checked clearly, and expanded over time without turning into a mess.